

计算机组成原理强化课考试

某字长为8位的计算机中，带符号整数采用补码表示， $a=-68, b=-80$ 。 a 和 b 分别存放在寄存器A和B中（可用 $A_7 \sim A_0$ 表示寄存器A的最高位到最低位，其他寄存器类似），请问：

- (1) 寄存器A和B中的内容分别为？
- (2) 若 $a+b$ 后的结果存放在寄存器C中，则寄存器C的内容是什么？运算结果是否正确？此时，符号标志SF、溢出标志OF、和零标志ZF各是什么？加法器最高位进位 F_{out} 是什么？
- (3) 若 $a-b$ 后的结果存放在寄存器D中，则寄存器D的内容是什么？运算结果是否正确？此时，符号标志SF、溢出标志OF、和零标志ZF各是什么？加法器最高位进位 F_{out} 是什么？
- (4) 请写出CF的计算公式。CF的含义是什么？
- (5) 请画出 8bit 加法器的示意图

解析：带符号整数的加减法溢出关注OF，

(1) $[-68]_{补} = [-1000100]_{补} = 1011\ 1100B = BCH$ 。 $[-80]_{补} = [-1010000]_{补} = 1011\ 0000B = B0H$ 。所以，寄存器A和B中的内容分别是BCH和B0H。

(2) $[x+y]_{补} = [x]_{补} + [y]_{补} = 1011\ 1100 + 1011\ 0000 = (1)\ 0110\ 1100 = 6CH$ ，最高位前面的一位1被丢弃，因此，寄存器C中的内容为6CH，对应的真值为+108，结果不正确。

- 溢出标志位OF可采用两种方法得到：

- 方法1：若两个加数的符号位相同，但与结果的符号位相异，则溢出。逻辑表达式如下

$$OF = \bar{A}_7 \bar{B}_7 C_7 + A_7 B_7 \bar{C}_7 \quad (3)$$

- 方法2：若最高位上的进位和次高位上的进位不同，则溢出。逻辑表达式如下

$$OF = F_{out} \oplus \text{次高位产生的进位} \quad (4)$$

- 运算结果的第一位（最高位）0为符号标志位SF，表示结果为正数（两个负数相加得到正数，显然错误）
- 运算结果不为全0，所以零标志ZF=0
- 加法器最高位产生的进位 F_{out} 为1

(3) $[x-y]_{补} = [x]_{补} + [-y]_{补} = 1011\ 1100 + 0101\ 0000 = (1)\ 0000\ 1100 = 0CH$ ，最高位前面的一位1被丢弃，因此，寄存器D中的内容为0CH，对应的真值为+12，结果正确。

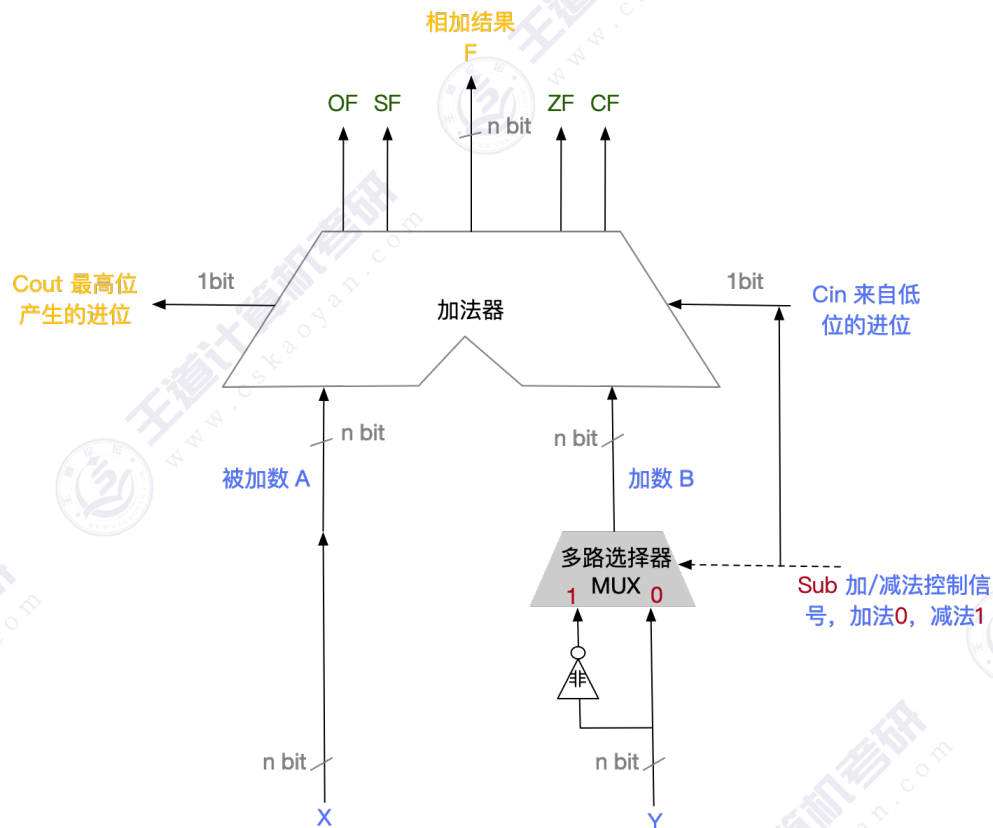
- OF=0。可用机算方法验证，也可以手算带入十进制。两个8bit带符号整数相减，真值为+12，未超出8bit补码可表示的范围，因此OF=0
- 运算结果的第一位（最高位）0为符号标志位SF，表示结果为正数
- 运算结果不为全0，所以零标志ZF=0
- 加法器最高位产生的进位 F_{out} 为1

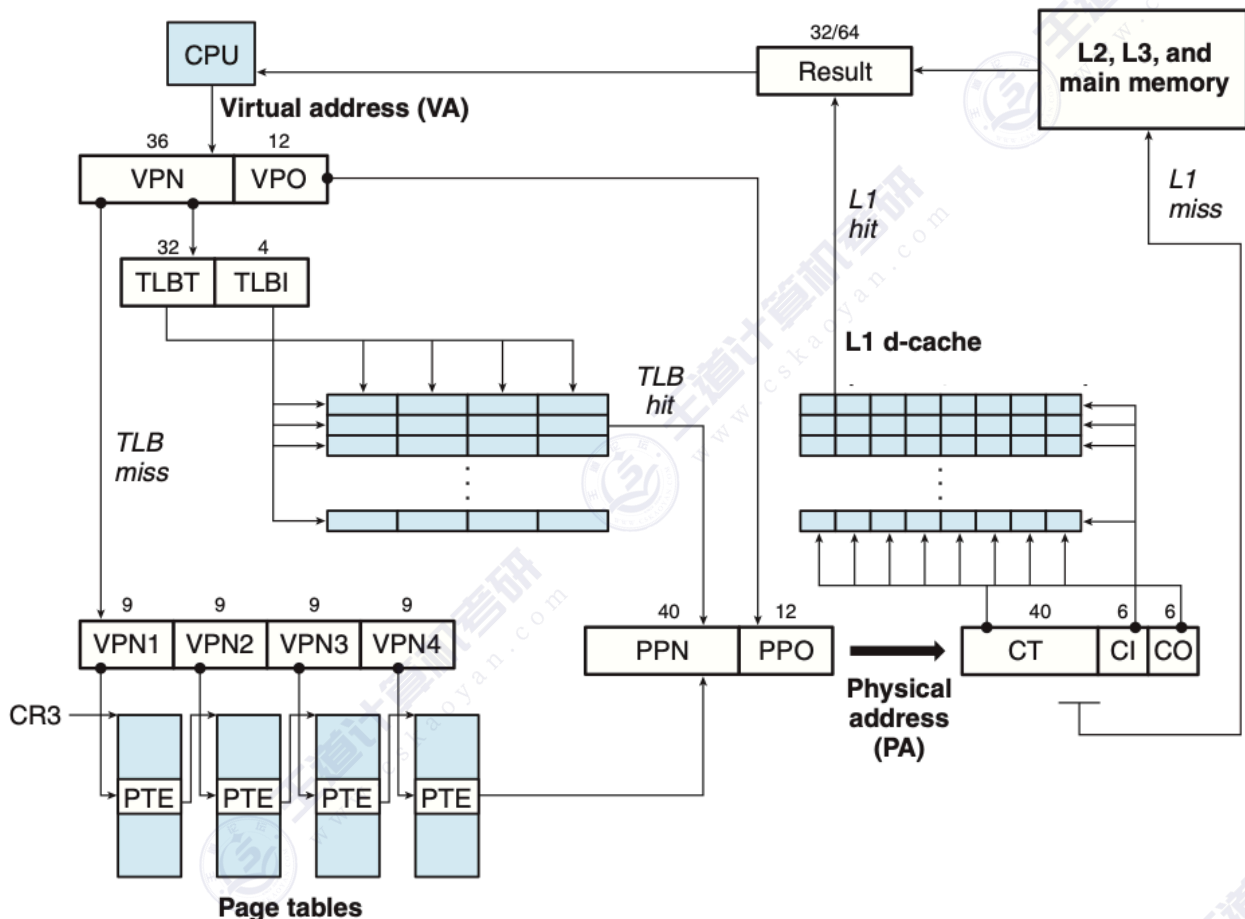
(4) $CF = F_{out} \oplus Sub$ （Sub=1表示这是减法，Sub=0表示这是加法， $\oplus$ 是异或）

CF标志位在带符号数加减法中无意义。在无符号数加减法中有意义，无符号数加法时，CF表示产生了进位；无符号数减法时，CF表示产生了借位。

- 有符号数的加减法运算，关注OF、SF、ZF 三个标志位信息，CF无意义。有符号数的运算结果是否溢出，通过OF来判断。
- 无符号数的加减法中，关注ZF、CF 两个标志位信息，OF、SF无意义。无符号数的运算结果是否溢出，通过CF来判断。

(5)





上面是 Intel Core i7 地址转换过程的简要示意图，已知该机按字节编址，请根据图示回答下面的问题：

- 1) 该CPU支持的虚拟地址空间大小是多少？物理地址空间大小是多少？
- 2) 该处理器支持几级页表？每个页面的大小是多少？每个页表项的大小是多少？
- 3) TLB采用什么映射方式？图示中 TLBT、TLBI 的作用是什么？请描述从虚拟地址转化为物理地址的过程。
- 4) Cache采用什么映射方式？每个Cache行的数据大小是多少？图示中 CT、CI、CO 的作用是什么？
- 5) 若Cache采用LRU替换策略，且采用写回法，请画出每个Cache行的结构，并计算图中 L1 d-Cache 的实际大小是多少？
- 6) 请描述根据物理地址访问图中Cache的过程。

解析：

1) 看图可知，虚拟地址VA总位数为 $36+12=48\text{bit}$ ，物理地址PA总位数为 $40+12=52\text{bit}$ 。

因此，虚拟地址空间大小为 $2^{48}\text{B} = 256\text{TB}$ ，物理地址空间大小为 $2^{52}\text{B} = 4\text{PT}$ 。

注：复习一下大纲要求掌握的数据单位 K、M、G、T、P、E、Z。依次递增，每增长一级就是乘 10^3 的关系，或者也可以是乘 2^{10} 的关系

2) 看图可知，该处理器支持4级页表。每个页面大小是 $2^{12}=4\text{KB}$ 。通常来说采用多级页表时，最理想的情况是每一级的页表大小不超过一个页面，而从图中可知，每一级的页号占9位，也就是说每一级页表有 2^9 个表项，因此每个页表项的大小为 $2^{12}/2^9 = 8\text{B}$ 。

3) TLB采用组相联映射, 具体来说, 4路组相联映射, 共 $2^4 = 16$ 组。TLBT, 即 TLB Tag, TLB标记信息。TLBI, 即TLB Index, 索引号, 也就是组相联映射中的“组号”, 从图中可知, TLBI共4bit, 说明总共有 $2^4 = 16$ 组。

地址转化过程如下:

1. 查TLB

1. 虚拟地址VA共48位, 取出前36位作为虚页号VPN, VPN再拆分为 32位TLBT、4位TLBI。
2. 用4位 TLBI 确定TLB组号, 一个分组中包含 4 个表项
3. 用32位TLBT 和分组中的 4个表项对比, TLBT匹配, 且有效位为1, 说明TLB命中, 直接得到物理页框号PPN (40bit)

2. 若TLB未命中, 则查内存中的页表

1. 虚拟地址的前36位作为虚页号VPN。再拆分为 9+9+9+9, 也就是4级页号
2. 根据每一级页号去查每一级页表。当然, 查页表的过程有可能会缺页, 如果缺页了还得调页
3. 查到最后一级页表, 就可以确定物理页框号PPN (40bit)

3. 将物理页框号PPN 和页内偏移量 VPO 拼接得到完整的物理地址 PA

4) Cache 采用组相联映射, 具体来说, 8路组相联映射, 共 $2^6=64$ 组

每个Cache行的数据大小是 $2^6=64B$

40位CT为Cache Tag, 用于判断Cache是否命中

6位CI为Cache Index, 用于表示Cache组号。其位数可以反映出总共有几个分组

6位CO为Cache Offset, 也就是Cache字块内地址 (块内偏移量)。其位数可以反映出每个Cache字块大小

5) 从图示可知, 该Cache采用8路组相联映射, 也就是说每个分组中有8个Cache行。若采用LRU替换策略, 得用 3bit 来标记组内各个Cache行。采用写回法 (write-back), 需要 1bit 作为脏位。每个Cache行的结构如下:

Tag标记	有效位	LRU替换信息	脏位	数据
40bit	1bit	3bit	1bit	64B

图中的L1 d-cache, 共 8路 $\times$ 64组 = 512 行, 每行的大小为 (40+1+3+1)bit + 64B。所以实际大小是 512 $\times$ [(40+1+3+1)bit + 64B] = 自己算吧('v')

6) 物理地址 PA 为40+12 = 52bit

- 就像图示那样, 拆分为 40bit CT, 6bit CI, 6bit CO
- 根据 6bit CI 找到对应 Cache分组, 分组内共8个Cache行

- 物理地址的前 40bit CT 与分组内的 8个Cache行的 Tag标记对比，如果Tag能匹配，且有效位为1，则 Cache命中，找到一个Cache行
- 根据字块内偏移量，6bit CO，从Cache行的64B数据中，访问数据
- 如果Cache不命中，则需要根据物理地址PA去访存（当然，实际上在 i7 处理器中，还有 L2级Cache、L3级Cache，都没命中才会访问主存）

拓展：下面这个图是 core i7 页表项的结构，每个页表项 64bit，即8B。看几个我们比较熟悉的东西：

63	62	52	51	12	11	9	8	7	6	5	4	3	2	1	0
XD	Unused	Page physical base addr				Unused	G	0	D	A	CD	WT	U/S	R/W	P=1

- P：有效位，或者翻译为存在位（Present），表示页面是否已经调入内存，p=0则缺页，p=1则不缺页。
- R/W：用 0/1 表示这个页面是否“只能读”，还是“可读可写”。比如存储常量的页面，就可以设置为只能读。
- U/S：用 0/1 表示这个页面在用户态下是否可访问，还是只有内核态下能访问。
- WT：用 0/1 表示这个页面的数据，对于Cache而言到底是“全写法 Write-Through”，还是“写回法 Write-back”。采用全写法意味着内存和Cache的数据是同步的，而写回法意味着内存里的数据可能不是最新的。
- A：访问位，表示最近有没有被访问过，可以被置换算法使用（回想一下Clock算法、改进型的Clock算法）
- D：脏位，表示页面信息是否被修改过。当这个页面的数据被“写”过，就脏了。
- 第 12~51 位，共40bit，表示下一级页框号。
- 第7个bit为0表示这一级页表大小为 4KB。该位为1表示这一级的页表大小为4MB。但是，只有顶级页表允许为4MB。也就是说，只有顶级页表的表项中，第7个bit可能为1；后面的几级页表的表项中，第7个bit一定是0。注意，当采用多级页表策略的时候，通常每一级页表的大小不超过一个页面。如果有必要，顶级页表大小可以超过1个页面。

有如下C语言过程：

```

1  int sum(int n)
2  {
3      int i=1;
4      int result=0;
5      for (i=1;i<=n;i++)
6          result+=i;
7      result*=2;
8      return result;
9  }
```

对应的部分汇编代码为：

```

1  1      movl    8(%ebp),%ecx
2  2      movl    $0,%eax
3  3      movl    $1,%edx
4  4      cmpl    %ecx,%edx
5  5      jg      L2
6  6  L1:  addl    %edx,%eax
7  7      addl    $1,%edx
8  8      cmpl    %ecx,%edx
9  9      jle     L1
10 10 L2:  sal     $1,%eax
11 11      ret

```

请问：

- (1) 该过程计算结果为？
- (2) eax、edx、ecx分别用于保存哪个变量？返回值在哪个寄存器中？
- (3) 请解释该汇编语言中，如何实现 for 循环？
- (4) 第10条汇编指令 sal 实现了算数左移，该指令对应C语言中哪句代码？还可以用别的指令实现同样的功能吗？
- (5) 哪几条指令会产生程序执行流的转移？

解析：

(1) result = n*(n+1)。（注：如果n的值太大，那么可能会溢出，返回结果可能出错）

(2) 由第2条指令知eax初值为0，即eax是result

由第3条指令知edx初值为1，即edx是i

由第8条指令知i与ecx做比较，可知ecx是n

返回result值，在eax中

(3) 参考：

1	1	movl	8(%ebp),%ecx	//将参数n的值从栈中取出，放入寄存器 ecx
2	2	movl	\$0,%eax	//变量 result 初始值为0，存在寄存器 eax
3	3	movl	\$1,%edx	//变量 i 初始值为1，存在寄存器 edx
4	4	cmpl	%ecx,%edx	//比较 i和n，本质上是在做 i-n。AT&T汇编语言，被减数在前，减数在后
5	5	jg	L2	//如果 i 比 n 大，则跳转到 L2
6	6	L1: addl	%edx,%eax	//实现 result = result + i
7	7	addl	\$1,%edx	//实现 i++
8	8	cmpl	%ecx,%edx	//比较 i和n
9	9	jle	L1	//如果 i比n小，或i=n，则跳转到 L1，否则顺序执行下一句
10	10	L2: sal	\$1,%eax	//算数左移两位，实现 result = result*2
11	11	ret		//返回上一层函数

(4) sal 指令对应 result*=2;

还可以用加法指令，实现 $(eax)+(eax) \rightarrow eax$

也可以用乘法指令，实现 $(eax) \times 2 \rightarrow eax$

(5) 第5、9行，条件转移指令，有可能改变PC的值；第11行 ret 指令，一定会改变PC的值。

注：本题中给大家的不是 x86 的汇编语言格式，而是 AT&T 的汇编语言格式。请复习 AT&T 与 x86 的区别：

	x86	AT&T	说明
寄存器的表示	eax	%eax	AT&T 格式的汇编语言，在寄存器名前边要加个百分号 %
操作数的顺序	OP 目的, 源操作数	OP 源, 目的操作数	x86的目的操作数在前，源操作数在后，AT&T刚好相反
常数的表示	10086	\$10086	AT&T中，表示常数时要在前边加一个美元符号，也许是这个公司很喜欢钱吧？另外，表示十六进制数的时候，x86是在十六进数后边加 h，而AT&T是在前边在 0x
访问内存中操作数时，如何指明操作数的长度	在要访问的内存地址之前注明操作数大小	在指令中隐含的描述操作数大小	x86的操作数大小在内存地址前边注明，如操作数大小为 1B/2B/4B分别可表示为：byte/word/dword ptr [var]。AT&T的操作数大小在指令中隐含表示，如操作数大小为 1B/2B/4B 分别在指令后面加一个字母：movb/movw/movl
内存偏移量的表示	[ebp+8]	8(%ebp)	x86中表示内存地址用方括号括起来，偏移量直接加进去。AT&T中表示内存地址用小括号括起来，偏移量写在小括号前面

某程序中有如下循环代码段P：“for(int i = 0; i < N; i++) sum+=A[i];”。假设编译时变量sum和i分别分配在寄存器R1和R2中。常量N在寄存器R6中，数组A的首地址在寄存器R3中。程序段P起始地址为0804 8100H，对应的汇编代码和机器代码如下表所示。

编号	地址	机器代码	汇编代码	注释
1	08048100H	00022080H	loop: sll R4, R2, 2	$(R2) \ll 2 \rightarrow R4$
2	08048104H	00083020H	add R4, R4, R3	$(R4) + (R3) \rightarrow R4$
3	08048108H	8C850000H	load R5, 0(R4)	$((R4) + 0) \rightarrow R5$
4	0804810CH	00250820H	add R1, R1, R5	$(R1) + (R5) \rightarrow R1$
5	08048110H	20420001H	add R2, R2, 1	$(R2) + 1 \rightarrow R2$
6	08048114H	1446FFFAH	bne R2, R6, loop	if(R2) != (R6) goto loop

执行上述代码的计算机M采用32位定长指令字，其中分支指令bne采用如下格式：

31 26	25 21	20 16	15 0
OP	Rs	Rd	OFFSET

OP为操作码；Rs和Rd为寄存器编号；OFFSET为偏移量，用补码表示。请回答下列问题，并说明理由。

- 1) M的存储器编址单位是什么？
- 2) 已知sll指令实现左移功能，数组A中每个元素占多少位？
- 3) 表中bne指令的OFFSET字段的值是多少？已知bne指令采用相对寻址方式，当前PC内容为bne指令地址，通过分析表中指令地址和bne指令内容，推断出bne指令的转移目标地址计算公式。
- 4) 若M采用如下“按序发射、按序完成”的5级指令流水线：IF（取值）、ID（译码及取数）、EXE（执行）、MEM（访存）、WB（写回寄存器），且硬件不采取任何转发措施，分支指令的执行均引起3个时钟周期的阻塞，则P中哪些指令的执行会由于数据相关而发生流水线阻塞？哪条指令的执行会发生控制冒险？为什么指令1的执行不会因为与指令5的数据相关而发生阻塞？
- 5) 模仿下图画出上述6条指令的指令流水线。

	时间单元													
指令	1	2	3	4	5	6	7	8	9	10	11	12	13	14
I_1	IF	ID	EX	M	WB									
I_2		IF	ID	EX	M	WB								
I_3			IF				ID	EX	M	WB				
I_4							IF				ID	EX	M	WB

解析：

这个题目是2014年真题，前四个小问就是真题本题，可参考王道书5.6大题4。第五个小问是一个扩展，参考答案如下：

若采用中断控制方式，则数据缓冲区每充满一次，就会产生一次中断，传送 d 字节，共产生中断 $d/4$ 次，每次中断处理时间开销为 t_3 。因此，每传送 d 字节，CPU 用于中断处理的时间总开销为 $(d/4)t_3$

综上，要使DMA控制方式效率更高，需要满足 $t_4 < (d/4)t_3$

通过这个题目，可以感受到，三种I/O方式没有绝对的孰优孰劣，要综合考虑设备速度、CPU处理一次IO的开销。